Day 35: Week 5 Review

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 5 Complete

Week 5 took the tuned XGBoost pipeline from Week 4 through four more stages: explaining its predictions (Days 28-29), understanding its behavior across feature ranges (Day 30), searching for better hyperparameters two different ways (Days 31-32), and finally making it durable and reachable (Days 33-34). Every score below was re-verified fresh while writing this review, not copied from memory.

1. Day 28 — SHAP Interpretability

> **Topic specification:** shap.TreeExplainer decomposes each individual prediction into a base value (the average model output) plus a per-feature contribution, all in log-odds space, summing exactly to that prediction's raw model output.

Explanation: Unlike global feature importance (which ranks features overall), SHAP explains one prediction at a time — which features pushed THIS passenger's probability up or down, and by how much. A sigmoid converts the summed log-odds back into the familiar 0-1 probability.

Real-world scenario: A loan applicant is denied by an automated model. SHAP values let the bank explain exactly which factors (income, credit history, debt ratio) drove that specific decision and by how much — a concrete, auditable explanation regulators increasingly require, not just a single opaque score.

Implementation:

```
explainer = shap.TreeExplainer(pipeline.named_steps["classifier"])
shap_values = explainer(X_transformed)
# base_value + shap_values.sum() == raw model margin
probability = 1 / (1 + np.exp(-(base_value + contributions.sum())))
```

Line-by-line explanation

• A real xgboost/shap version incompatibility surfaced here: xgboost >=2.0 serializes base_score as a bracketed string like `'[3.131313E-1]'`, which crashes shap's internal float() conversion.

• Fixed with a targeted monkeypatch on shap's UBJSON decoding step, root-caused by reading shap's own source rather than guessing — reused verbatim in every SHAP-based day afterward.

> **Why choose this?** Per-prediction explanations build trust and satisfy audit requirements that a global importance ranking alone cannot — essential wherever an individual decision needs to be justified, not just the model's average behavior.

> **Why NOT this (tradeoffs)?** TreeExplainer's exact, fast computation is specific to tree-based models — this approach doesn't transfer to arbitrary model types without switching to a slower, approximate SHAP explainer variant.

2. Day 29 — Visual SHAP

Topic specification: Four SHAP plot types turn Day 28's numbers into pictures: a global bar chart (mean |SHAP| per feature), a beeswarm (per-feature distribution across all passengers), dependence plots (one feature's SHAP value vs. its raw value), and a waterfall (one passenger's full explanation).

Explanation: Each visualization answers a different question. The bar chart asks 'which features matter most overall'. The beeswarm adds 'and in which direction, for how many passengers'. Dependence plots ask 'does this feature's effect change depending on its value'. The waterfall answers 'why did this one passenger get this one prediction'.

Real-world scenario: A model risk team reviewing a newly deployed model uses the beeswarm plot to confirm Sex and Pclass drive most predictions in the expected direction, then pulls waterfall plots for a handful of edge-case predictions to manually sanity-check before signing off on production deployment.

Implementation:

```
shap.plots.bar(shap_values, show=False)
shap.plots.beeswarm(shap_values, show=False)
shap.plots.scatter(shap_values[:, "Fare"], show=False)
shap.plots.waterfall(shap_values[0], show=False)
```

Line-by-line explanation

• The first PDF draft for this day shipped with no embedded charts at all — caught directly: "but i dont see any visual chart in the result".

• Fixed by generating real PNGs into a scratch assets folder and adding a figure() helper (PIL for proportional sizing, ReportLab's Image flowable for embedding) — every PDF from this day forward embeds actual generated images.

Why choose this? Visualizations make patterns immediately legible that would take many individual SHAP value lookups to notice manually — the beeswarm alone often reveals more than a printed table of importance scores ever could.

Why NOT this (tradeoffs)? show=False is required in a headless sandbox (no display backend) — a subtlety easy to trip over, and each plot type answers a genuinely different question, so no single chart replaces the full set.

3. Day 30 — Partial Dependence and ICE

Topic specification: PartialDependenceDisplay and partial_dependence() show how the model's average prediction changes as ONE feature varies across its range, holding all other features fixed — PDP shows the average curve, ICE shows individual passenger curves.

Explanation: SHAP explains individual predictions relative to a baseline; PDP/ICE instead show the model's learned RELATIONSHIP with a feature — does survival probability rise smoothly with Fare,

or plateau, or have a threshold? This is a different, complementary question.

Real-world scenario: A pricing team wants to know whether a demand model's relationship with price is smooth or has a cliff — a PDP curve reveals that shape directly, informing where a price change might cause a disproportionate swing in predicted demand versus where it's safe.

Implementation:

```
def build_pdp_reference(X):
    X_ref = X.copy()
    for col in ["Age", "Fare"]:
        if X_ref[col].isna().any():
            X_ref[col] = X_ref[col].fillna(X_ref[col].median())
    return X_ref

X_train_pdp = build_pdp_reference(X_train)
PartialDependenceDisplay.from_estimator(pipeline, X_train_pdp, ["Age"])
```

Line-by-line explanation

• A genuine sklearn.inspection bug surfaced here, caught from a user screenshot of blank charts: the grid-building step uses np.percentile on the RAW column BEFORE the pipeline's own imputer runs — NaN-unsafe, silently producing an all-NaN grid on Age's ~20% missing rate.

• Fixed with a pre-imputed reference frame used only for grid construction; predictions still run through the real, unmodified pipeline — verified both numerically (grid.min()=9.9, grid.max()=51.6) and visually.

Why choose this? PDP/ICE reveal the shape of a model's learned relationship with a feature directly, which SHAP's per-prediction attributions don't show as clearly on their own.

Why NOT this (tradeoffs)? This was the week's most subtle bug: no exception was raised, just silently blank output — a strong reminder that a script running without errors is not the same as a script producing correct results.

4. Day 31 — RandomizedSearchCV

Topic specification: RandomizedSearchCV samples a fixed number of random combinations from continuous parameter distributions (scipy's randint, uniform, loguniform), instead of exhaustively trying every combination in a fixed grid.

Explanation: A genuinely thorough 7-parameter grid (3,888 combinations) would require 19,440 model fits at cv=5 — far too slow. RandomizedSearchCV instead samples n_iter combinations from continuous distributions, covering far more of the space per fit spent, at the cost of no guarantee of hitting the single best point exactly.

Real-world scenario: A team with a 2-hour nightly retraining window can't exhaustively grid-search 7 hyperparameters, but can run RandomizedSearchCV with a fixed n_iter that fits inside that window every night, trading a guaranteed-optimal answer for one that reliably finishes on time.

Implementation:

```
param_distributions = {
    "classifier__n_estimators": randint(100, 400),
    "classifier__learning_rate": loguniform(0.01, 0.3),
    "classifier__max_depth": randint(2, 8),
    ...
}
search = RandomizedSearchCV(pipeline, param_distributions, n_iter=18, cv=5)
```

Actual output:

search	space	fits	time (s)	best CV score
GridSearchCV	3 params, 18 fixed combos	90	2.0	0.746
RandomizedSearchCV	7 params, continuous	90	1.7	0.744

Re-verified fresh while writing this review — both scripts rerun end to end.

Line-by-line explanation

• `loguniform(0.01, 0.3)` — samples learning_rate on a log scale, so small values (0.01, 0.02, 0.05) get proportionally as much sampling density as large ones (0.1, 0.2, 0.3), matching how learning rate actually affects training.

• Same 90-fit budget, nearly identical best score (0.746 vs 0.744) — the real payoff isn't a better score here, it's that the random search covered a 7-dimensional continuous space the grid search structurally could not.

Why choose this? Scales to far more hyperparameters and continuous ranges without the combinatorial explosion a grid suffers from — the practical default once a search space grows past 3-4 parameters.

Why NOT this (tradeoffs)? No guarantee of finding the exact grid optimum within a fixed budget — verified here: the small, focused grid actually scored marginally higher (0.746 vs 0.744) on this particular budget and space.

5. Day 32 — Optuna (Bayesian Optimization)

Topic specification: Optuna's TPE (Tree-structured Parzen Estimator) sampler builds a running model of which hyperparameter regions tend to score well, and biases each new trial's suggestion toward those regions — using every prior trial's result to inform the next one.

Explanation: RandomizedSearchCV samples every trial independently — trial 15 knows nothing about trials 1-14, even if several already scored badly in the same region. Optuna's TPE sampler actively learns from completed trials, which should, on average, find better results than blind sampling within the same fit budget.

Real-world scenario: A research team with a $10,000 compute budget for hyperparameter search on a large language model uses Bayesian optimization instead of random search specifically because every wasted trial is expensive — an adaptive search that learns from its own history

extracts more value per dollar spent.

Implementation:

```
def objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 100, 400),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        ...
    }
    return cross_val_score(build_pipeline(params), X, y, cv=5).mean()

study = optuna.create_study(direction="maximize", sampler=TPESampler(seed=42))
study.optimize(objective, n_trials=18)
```

Actual output:

method	strategy	time (s)	best CV score
GridSearchCV	exhaustive, 3 params	2.1	0.746
RandomizedSearchCV	independent random, 7 params	1.8	0.744
Optuna (TPE)	adaptive Bayesian, 7 params	1.9	0.747

Re-verified fresh — same 90-fit budget across all three, rerun end to end.

Line-by-line explanation

• Verified: Optuna's TPE sampler found the best score of all three methods (0.747) on the identical 90-fit budget — the expected direction for an adaptive sampler, though not a large margin here.

• optuna.importance.get_param_importances ranked learning_rate as the most influential hyperparameter (40.1% of the surrogate model's explained variance) — a different kind of importance than Day 23/25's feature importance, ranking hyperparameters' effect on score, not features' effect on prediction.

Why choose this? Learning from prior trials is a strictly more informed strategy than sampling blind, and the parameter-importance analysis is a genuinely useful byproduct random search doesn't provide.

Why NOT this (tradeoffs)? The margin over random search was modest here (0.747 vs 0.744) — Bayesian optimization's advantage compounds with much larger trial budgets and higher-dimensional spaces; at 18 trials the gap is real but not dramatic.

6. Day 33 — Model Persistence

Topic specification: joblib.dump/joblib.load save and load the ENTIRE fitted pipeline as one object — engineer, preprocessor, and classifier together — proven necessary by deliberately showing what happens when only the classifier is saved.

Explanation: A trained classifier alone has no knowledge of how to turn raw strings and missing values into the numeric, encoded, scaled input it was actually trained on. Only the whole pipeline, saved as one object, remembers every preprocessing step needed to go from raw to prediction.

Real-world scenario: A production incident traced back to a junior engineer saving only pipeline.named_steps['classifier'] to save disk space — every live prediction request threw a type error until someone realized the preprocessing steps had never been saved at all.

Implementation:

```
joblib.dump(pipeline, "titanic_pipeline.joblib")
metadata = {"sklearn_version": sklearn.__version__, "xgboost_version": xgb.__version__,
            "cv_accuracy_mean": round(float(cv_score), 4), ...}
json.dump(metadata, open("titanic_pipeline_metadata.json", "w"))
```

Actual output:

```
Pipeline fitted. 5-fold CV accuracy: 0.719 (+/- 0.035)
Saved: titanic_pipeline.joblib (225.8 KB)
```

Re-verified fresh while writing this review.

Line-by-line explanation

• Worth noticing, found while re-running this pipeline for the review: its CV accuracy (0.719) is lower than what Day 31/32's searches found (0.744-0.747) — the persisted pipeline uses hyperparameters fixed manually earlier in the week, not the literal best values the search days discovered.

• This is a genuine, common real-world gap: a search finding a better configuration doesn't automatically update the production pipeline — that sync has to happen as a deliberate step, easy to forget in practice.

• A None-vs-np.nan bug was also caught here: Python's None doesn't match SimpleImputer's default missing_values=np.nan check on object columns — fixed in the test data construction.

Why choose this? Persistence separates the (expensive) training step from the (cheap) prediction step permanently — the foundation every later deployment step depends on.

Why NOT this (tradeoffs)? A persisted file is frozen at whatever it learned at save time, including whatever hyperparameters happened to be hard-coded then — as demonstrated here, it does not automatically benefit from better parameters found by later experimentation.

7. Day 34 — Serving with FastAPI

Topic specification: A FastAPI app loads Day 33's saved pipeline exactly once at startup (via a lifespan context manager), validates every incoming request against a Pydantic schema, and returns a live prediction from a POST endpoint.

Explanation: This is the step that makes a persisted model actually reachable by other systems — a mobile app, a website backend, another service — over a standard HTTP interface, without any of those callers needing Python or the model file installed.

Real-world scenario: A customer-facing web application, written in an entirely different language, sends applicant details to this FastAPI endpoint and gets a prediction back as JSON — the two systems only need to agree on an HTTP contract, nothing more.

Implementation:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    model_state["pipeline"] = joblib.load(MODEL_PATH)
    yield

@app.post("/predict", response_model=PredictionOutput)
def predict(passenger: PassengerInput):
    row = pd.DataFrame([passenger.model_dump()])
    row = row.where(row.notna(), np.nan)
    return PredictionOutput(survived=..., survival_probability=...)
```

Line-by-line explanation

• Two real gotchas resurfaced here, both caught by directly reproducing them: an unpicklable FunctionTransformer reference (fixed by importing add_family_size from its original module), and Day 33's None-vs-NaN bug, now triggered by real JSON null values from API requests instead of hand-built test data.

• TestClient exercised the whole app in-process, including the lifespan startup event, without needing a real running server — the same verification approach used all week.

Why choose this? This is the payoff of everything from Days 28-33 — an explainable, tuned, persisted model that other systems can actually call.

Why NOT this (tradeoffs)? A minimal single-process API has no monitoring, no horizontal scaling, and reloads the model only on restart — genuinely production-grade deployment needs more than this lesson covers, which is intentional: this is the first piece, not the whole system.

Interview Preparation — Technical Questions (Week 5)

Q: What's the core difference between what SHAP explains and what a partial dependence plot explains?

A: SHAP explains one specific prediction — how much each feature pushed that individual output away from the average. A PDP explains the model's average learned relationship with one feature across its whole range, independent of any single prediction. They answer complementary, not competing, questions.

Q: Why did Day 30's PDP charts render blank with no error raised?

A: sklearn.inspection builds its prediction grid using np.percentile on the raw, pre-preprocessing column — which is not NaN-safe. With ~20% missing Age values, that produced an all-NaN grid silently, since np.percentile doesn't raise on NaN input by default, it just propagates NaN through.

Q: Why does RandomizedSearchCV sometimes score worse than a well-chosen small grid on the same fit budget, as observed in this week's rerun (0.744 vs 0.746)?

A: Random sampling from a large continuous space has no guarantee of landing near the single best point a focused, well-chosen small grid might hit directly — the tradeoff is coverage of a much larger space versus precision on a small one, and coverage doesn't always win on a fixed, modest budget.

Q: What specifically does Optuna's TPE sampler do differently from RandomizedSearchCV's sampling?

A: TPE builds a probabilistic model of which hyperparameter regions produced good versus bad scores from all completed trials, then samples the next trial preferentially from the promising regions. RandomizedSearchCV samples every trial independently from the same fixed distributions regardless of prior results.

Q: Why did the persisted Day 33 pipeline score lower (0.719) than Day 31/32's search results (0.744-0.747) despite using 'final tuned settings'?

A: The persisted pipeline's hyperparameters were fixed manually at a point earlier in the week's development, not automatically updated with the specific best values Day 31/32's searches later found — a real, common gap between experimentation and what actually gets shipped.

Q: Why does saving only pipeline.named_steps['classifier'] fail when used on raw data?

A: The classifier was trained on fully numeric, imputed, scaled, encoded arrays — it has no mechanism to interpret a raw string column or a missing value. Only the full pipeline, including its fitted preprocessing steps, remembers how to perform that transformation.

Q: Why did the FastAPI app need to import add_family_size from day33_model_persistence even though it's never called directly in day34's code?

A: joblib/pickle stores a reference to that function (module + name), not its source code. Unpickling the saved pipeline requires Python to successfully import that exact name from that exact module — the import's only purpose is making that resolution possible.

Q: Why did the same None-vs-NaN bug from Day 33 reappear in Day 34's API context?

A: Pydantic's Optional fields deserialize a JSON null into Python's None, which SimpleImputer's default missing_values=np.nan check does not recognize on object-dtype columns — the identical root cause as Day 33's hand-built test data, now triggered by real API requests instead.

Interview Preparation — Theoretical Questions (Week 5)

Q: How does Week 5's progression — explain, understand relationships, tune, persist, serve — reflect the actual lifecycle of a model in production, beyond just training it?

A: Training a model that scores well is only the first step. Explaining it builds trust and satisfies audit needs; understanding feature relationships catches unexpected behavior; tuning improves it within a fixed budget; persisting and serving make it durable and reachable. A model that only exists as a notebook cell is not yet a usable system — Week 5 traces exactly what closes that gap.

Q: Why is it significant that both Day 30's PDP bug and Day 33/34's None-vs-NaN bug produced no exceptions, just silently wrong or degraded results?

A: Bugs that raise loud errors get caught immediately by definition. Bugs that fail silently — wrong output with no crash — are far more dangerous precisely because nothing signals something is wrong; they require actively verifying results, not just checking that code ran without error, which is why visual and numeric verification became a deliberate practice this week.

Q: Why does the gap between Day 31/32's search results and Day 33's persisted pipeline's actual hyperparameters illustrate a broader organizational challenge in machine learning teams?

A: Experimentation (searches, notebooks, one-off scripts) and production (the pipeline that actually gets deployed) are often maintained separately, and nothing automatically synchronizes them. Without a deliberate process — like a model registry that tracks which search produced which deployed model — better results found in experimentation can silently never reach production, exactly as observed by rerunning this week's own scripts.

Q: Why does an adaptive method like Optuna's TPE sampler not guarantee a better result than random search on every run, despite using more information?

A: TPE's advantage is statistical, not deterministic — it biases sampling toward historically promising regions, but a fixed, small trial budget (18 trials here) still carries real sampling variance, and the true optimum may not be captured by either method's limited exploration. The advantage compounds and becomes more reliable with larger budgets and higher-dimensional spaces than this week's controlled comparison used.

Q: Why does SHAP's per-prediction attribution matter more in regulated domains than a simple accuracy metric?

A: An accuracy metric answers 'how often is the model right', which says nothing about why any single decision was made. Regulated domains (lending, hiring, healthcare) increasingly require justifying individual decisions, not just demonstrating aggregate performance — exactly the question SHAP's per-prediction decomposition answers and a global metric cannot.

Q: Why does persisting the WHOLE pipeline, rather than the classifier plus separately documented preprocessing steps, reduce a specific category of production risk?

A: Separately documented preprocessing creates two independent sources of truth — the pipeline code and the documentation describing it — that can silently drift out of sync as either changes over time. Persisting the whole pipeline as one object eliminates that entire category of bug by construction, since there's only one artifact to keep correct.

Q: Why does deploying behind a minimal single-process API (Day 34) represent a deliberate starting point rather than a finished production system?

A: A single process with no monitoring, no horizontal scaling, and no hot-reload mechanism works correctly but doesn't handle the operational realities of real traffic: failures, load spikes, or the need to update the model without downtime. Building the simplest correct version first, then layering resilience on top, is a deliberate engineering sequence, not a shortcut.

Key Takeaway and What's Next

Key takeaway: Week 5 turned a tuned model into a real system: explainable (SHAP), understood (PDP/ICE), optimized within a fixed budget two different ways (random search, Bayesian optimization), durable (persistence), and reachable (FastAPI). Three genuine bugs were caught and fixed along the way — a library version mismatch, a NaN-unsafe grid calculation, and a None-vs-NaN gotcha that recurred in two different contexts — each one root-caused directly rather than worked around.

Week 6 preview

• `Dataset switch` — Titanic carried the roadmap through the end of Week 5; Week 6 moves to a dataset better suited to neural networks.

• `Deep learning foundations` — neural network fundamentals begin replacing XGBoost as the model architecture being built, trained, and eventually persisted and served the same way this week's pipeline was.

• `Same rigor, new tools` — the habits from Week 5 (verify actual output, don't trust a script running without errors, root-cause real bugs instead of guessing) carry forward unchanged into a new model family.